



北京航空航天大学
BEIHANG UNIVERSITY

MPI 编程实验报告

《并行程序设计 A》课程报告

学 院： 软件学院

班 级： ZY25211

姓 名： 刘益洲

学 号： ZY2521110

目录

一、 实验环境	1
二、 实验内容与方法	2
(一) 实验内容	2
(二) 实验方法	2
三、 实验过程	5
(一) MPI 块矩阵乘法实现要点	5
(二) 计时与五轮基准实验	6
(三) 时间线分析实验	6
(四) 程序编译与运行	7
四、 实验结果与分析	7
(一) 五轮基准实验结果	7
(二) 时间线实验与计时段行为	9

一、实验环境

本实验在表 1、表 2 所列环境下完成；表中数值由集群配置说明及容器内 `lscpu`、`nproc`、`uname` 等命令整理得到。

表 1 实验硬件与系统环境

名称	版本或配置
集群	Kubernetes
工作负载	Pod
containerd	2.2.1
Kubelet	v1.35.2
Pod CPU 配额	70 核
Pod 内存配额	64 GiB
CPU 绑核	未开启
处理器	Intel Xeon Gold 6148 @ 2.40 GHz (2 路, 20 核/路, HT)
逻辑 CPU	80
架构	x86_64
Ubuntu	24.04.1 LTS
Linux 内核	5.15.0-170-generic
Locale	C.UTF-8

MPI 程序所用编译与运行环境见表 2；本实验仅使用 MPI，不使用 OpenMP。

表 2 MPI 与编译工具环境

名称	版本或配置
GCC	13.3.0
G++	13.3.0
Open MPI	4.1.6
mpicc / mpicxx	Open MPI 4.1.6 (GCC 13.3.0)
mpirun	4.1.6
mpirun 启动选项	--use-hwthread-cpus
GNU Make	4.3
glibc	2.39
编译优化	-O3

二、实验内容与方法

(一) 实验内容

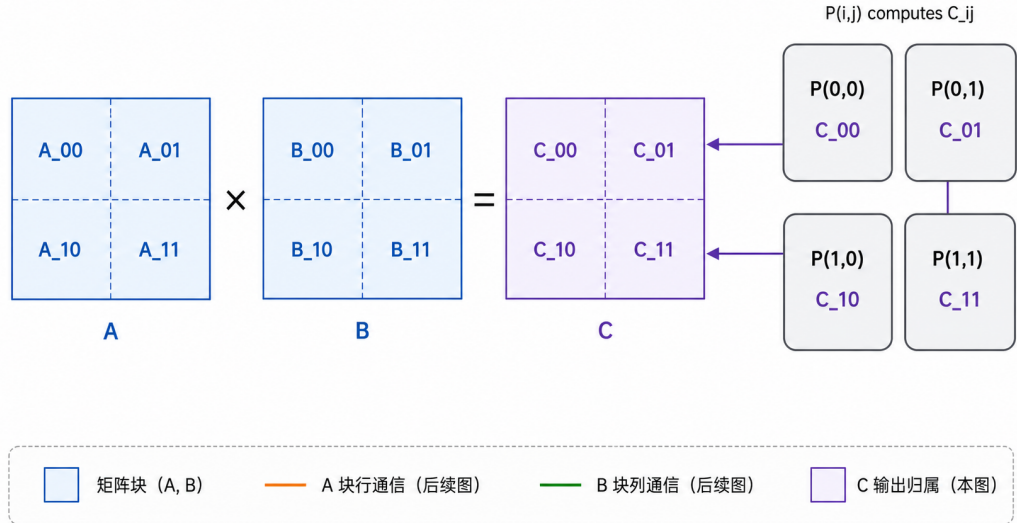
第3次作业要求使用 MPI 实现矩阵并行乘法，编程语言为 C/C++。方阵规模 $N \times N$ 满足 $N \geq 8000$ ，矩阵元素为双精度浮点数；按三矩阵各占用约 N^2 个双精度元素估算， $N = 8000$ 时单矩阵约 512 MiB，三矩阵合计约 1.5 GiB 量级，与作业说明中常用的约 480 MB 估算口径一致。实验需完成并程序，并与串行实现对比，给出加速比；统计计算时间时应排除矩阵初始化、文件 I/O 等数据准备时间。此外需考察并程序的可扩展性，例如改变 MPI 进程数、关注数据划分带来的局部性等因素。

(二) 实验方法

本实验在单节点、多进程下用 MPI 组织并行，通过 `mpirun -np p` 启动进程，运行环境为 Open MPI 4.1.6。将 $N \times N$ 矩阵 A, B, C 在行列方向各划分为 $P_r \times P_c$ 个子块，其中 $P_r P_c = p$ ，且 p 分解为尽量接近方阵的进程网格；记块下标为 i, j, k ，子块 $A_{i,k}$ 、 $B_{k,j}$ 、 $C_{i,j}$ 的规模随 N 与余数分配略有差别，但满足块矩阵乘法，与作业示意图一致：

$$C_{i,j} = \sum_{k=0}^{P_c-1} A_{i,k} B_{k,j}.$$

进程按二维坐标 (i, j) 编号， $\text{rank} = i \cdot P_c + j$ ，负责累加本地 $C_{i,j}$ 。对每个 k ，自进程 (i, k) 接收 $A_{i,k}$ 、自 (k, j) 接收 $B_{k,j}$ ；若该进程即为子块拥有者，则直接访问本地数据。完成子块乘加后写入 $C_{i,j}$ 。乘加结束后，各进程将 $C_{i,j}$ 发往根进程，在 rank 0 上拼装为完整的 $N \times N$ 矩阵 C ，从而构成完整的并行矩阵乘法流程。 $p = 1$ 时无进程间通信。本地子块内部仍用朴素三重循环，不在子块内再做 cache 分块。图 1 以 2×2 进程网格为例示意 A, B, C 的子块划分及进程与子块的对应关系。

图 1 矩阵二维块划分与进程—子块对应关系示意 (2×2 网格示例)

就通信而言，计时段内依次经历子块分发、按 k 轮的块乘加通信与结果汇聚三阶段，划分与是否计入 T_p 见表 3。仅根进程上 A, B 的伪随机初始化在 MPI_Wtime 之前完成，以符合作业“排除数据准备”的要求。图 2 概括了各阶段进程间的数据移动及最终汇聚得到全局 C 的过程；本程序在乘加阶段采用按 k 轮的点对点收发，而非环移算法。

表 3 MPI 矩阵乘法的进程间通信阶段

阶段	是否计入 T_p	内容与原语
数据初始化	否	根进程生成 A, B 元素值（在 MPI_Wtime 之前）
子块分发	是	$A_{i,k}$ 发往 (i,k) , $B_{k,j}$ 发往 (k,j) ($\text{MPI_Send}/\text{MPI_Recv}$)
块乘加通信	是	对每个 k : 交换 $A_{i,k}$ 、 $B_{k,j}$ 并本地累加 $C_{i,j}$
结果汇聚	是	各进程将 $C_{i,j}$ 发往 rank 0, 拼装为全局 C

程序未使用 $\text{MPI_Isend}/\text{MPI_Irecv}$ 等非阻塞接口，通信与本地乘加之间也不做流水线重叠；各阶段依靠阻塞式点对点收发推进。对每个 k ，拥有 $A_{i,k}$ 的进程先向同行其余进程 MPI_Send ，需要该子块的进程 MPI_Recv ； $B_{k,j}$ 同理。接收调用在匹配消息到达前一直阻塞，故时间线中的 recv 条带包含等待对方发送的时间，而不是“未记录的空白”。计时段内仅在 **scatter** 之前调用一次 MPI_Barrier ，配合 MPI_Bcast

广播 rank 0 的 `MPI_Wtime` 作为全局时间原点；scatter、各轮 k 与 gather 之间不再加屏障，各 rank 可在阻塞收发匹配的前提下先后推进，墙钟 T_p 由最慢进程决定。开启 `MATMUL_EVENTS` 写事件 CSV 时，在计时段结束之后另有屏障，用于保证目录创建与写盘完成，不计入 T_p 。

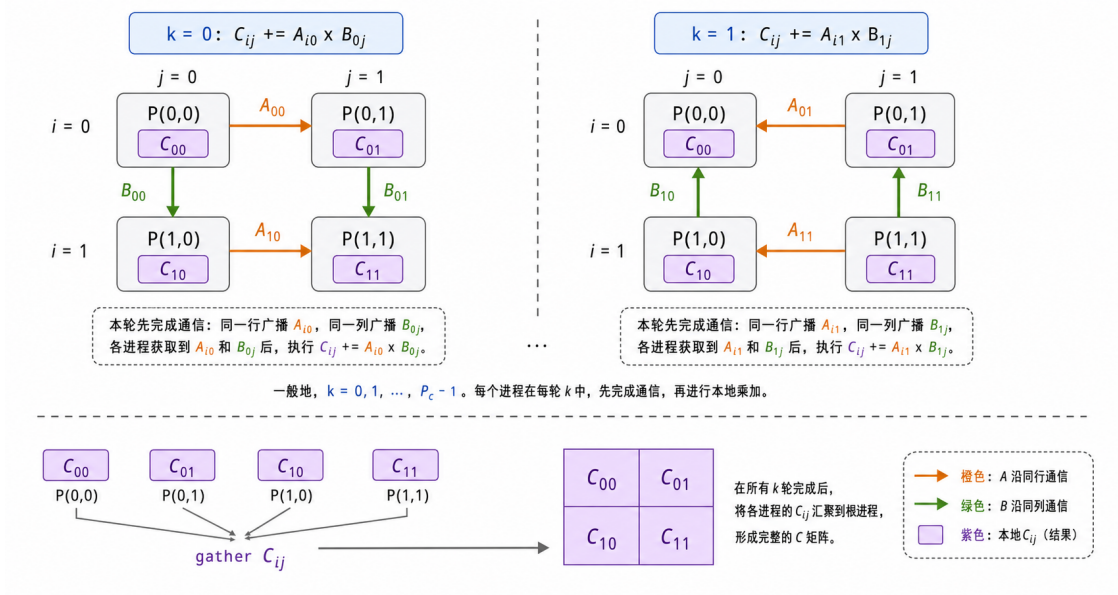


图 2 进程间通信与结果汇聚示意 (2×2 网格示例)

在 Pod 申请 70 CPU、64 GiB 内存且未绑核的条件下，固定矩阵规模、编译选项与计时口径，仅改变 MPI 进程数以考察可扩展性，具体取值如表 4。

表 4 矩阵乘法实验因素设计

名称	取值或说明
矩阵阶数 N	8000 (满足 $N \geq 8000$ ，三矩阵约 1.5 GiB)
进程数 p	64, 32, 16, 8, 4, 2, 1 (从大到小依次测试)
进程网格	$P_r \times P_c = p$, $P_r \geq P_c$ ，因子分解尽量接近方阵
mpirun 选项	--use-hwthread-cpus (见下文)
并行策略	二维块划分, $C_{i,j} = \sum_k A_{i,k} B_{k,j}$
数据类型	double, 行主序存储
数据生成	根进程确定性初始化 (MPI_Wtime 之前)
计时区间	子块分发 + 各 k 子块收发与乘加 + C 汇聚
排除项	矩阵元素初始化、文件 I/O 与写盘
重复次数 (基准)	每个 (N, p) 共 5 轮 (rep=1,...,5)，对 time_sec 取算术平均
时间线实验	与基准分离：每个 p 单独运行 1 次 (MATMUL_EVENTS=1)，见下文
编译选项	mpicc -O3 -std=c11

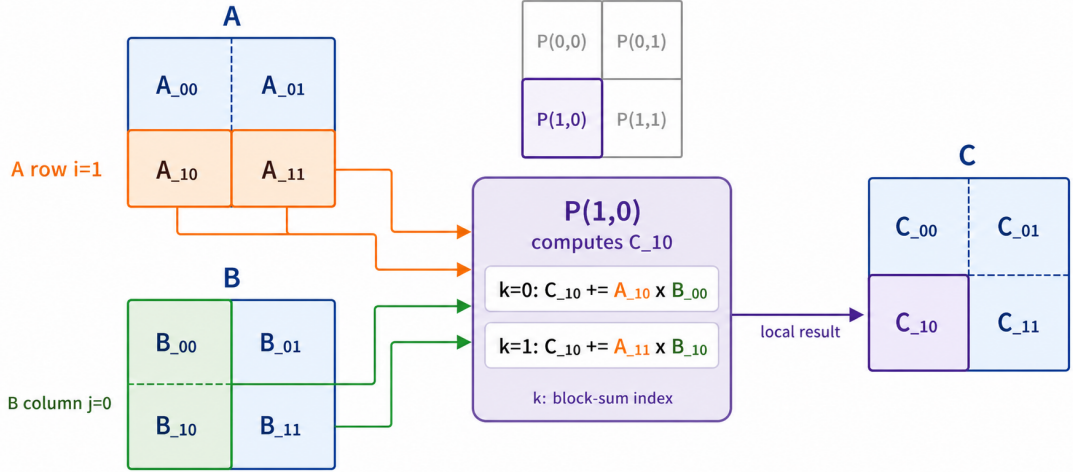
进程数取 2 的幂直至 64, 例如 $p = 8$ 时网格为 2×4 ; $p = 1$ 作为 T_1 。Pod 申请 70 CPU 是为减少同节点其它任务干扰, 实验进程数最大仍为 64。各进程常驻内存约为一个 $C_{i,j}$ 子块, 乘加阶段另需缓冲区存放当前 $A_{i,k}$ 、 $B_{k,j}$, 总占用随 p 增大而分散。 $p = 2$ 时因子分解为 $P_r = 2, P_c = 1$, 网格退化为 2×1 , 与 $p \geq 4$ 时的近方阵划分不同, 后文单独说明。实验初期直接执行 `mpirun -np 64 ./matmul_mpi 8000` 时, Open MPI 曾报错: *not enough slots available*。Open MPI 用 slot 估计本机可启动的进程数; 默认常按物理核数记账, 本机约 40, 低于容器内可见的逻辑 CPU 数 80。从硬件上讲, 一个逻辑 CPU 通常可运行一个 MPI 进程; 因此本实验在 `mpirun` 中增加 `--use-hwthread-cpus`, 按逻辑 CPU 数分配 slot, 使 $p \leq 64$ 的测试可以正常启动。基准脚本与手工运行均采用该选项, 见表 4 与表 2。

记第 p 个配置五轮基准实验的平均耗时为 T_p , 加速比 $S_p = T_1/T_p$, 并行效率 $E_p = S_p/p$, 其中 T_1 为 $p = 1$ 时的平均 `time_sec`。基准测时先 `make` 生成 `matmul_mpi`, 再依次执行 `./bench_matmul_mpi.sh 1 --fresh` 与 `2,...,5`; 每轮脚本内按 $p = 64, 32, \dots, 1$ 测完全部进程数, `mpirun` 使用 `--use-hwthread-cpus`, 默认不开启事件记录, 仅向 `results/matmul_bench.csv` 追加一行总时间, 同一轮内相邻两次 `mpirun` 间隔 10s。时间线分析另与基准分开进行, 以免大量事件记录拖慢批量实验: 使用 `./bench_matmul_timeline.sh` 对每个 p 单独运行一次, 并设置 `MATMUL_EVENTS=1`。各 MPI 进程将计时段内每次 `pack/send/recv/compute/alloc` 记为一条事件, 写入 `results/events/N8000_p<p>_rep1_rank<r>.csv`; 同时在 `results/matmul_bench` 记录该次运行的 `time_sec`, 用于与基准对比 `profiling` 引入的开销。事件时间戳以 rank 0 在 `scatter` 之前经 `MPI_Barrier` 对齐后的 `MPI_Wtime` 为原点, 便于绘制全体进程 Gantt 图; 写 CSV 在计时段结束之后进行, 不计入 T_p 。可用 `plot_timeline.py` 由事件目录生成 `plots/` 下图。

三、实验过程

(一) MPI 块矩阵乘法实现要点

程序将 p 分解为 $P_r \times P_c$, 建立逻辑坐标 (`proc_row`, `proc_col`)。根进程在计时外初始化 A, B 后, `MPI_Wtime` 计时区间内依次完成: 把 $A_{i,k}$ 、 $B_{k,j}$ 分发到各拥有进程; 对每个 $k \in [0, P_c)$ 交换子块并累加 $C_{i,j}$; 将各 $C_{i,j}$ 汇聚为 rank 0 上的完整 C 。图 3 以进程 $P(1,0)$ 为例说明在 $k = 0, 1, \dots$ 各轮中本地所需的 $A_{i,k}$ 、 $B_{k,j}$ 子块及累加得到 $C_{i,j}$ 的过程。子块乘法核心如代码 1 所示。

图 3 单进程在按 k 轮乘加中的本地子块依赖示意 (2×2 网格示例)

```

1 for (int ii = 0; ii < rows_a; ii++)
2   for (int jj = 0; jj < cols_b; jj++) {
3     double sum = C_local[ii * cols_b + jj];
4     for (int kk = 0; kk < cols_a; kk++)
5       sum += A_sub[ii * cols_a + kk] * B_sub[kk * cols_b + jj];
6     C_local[ii * cols_b + jj] = sum;
7   }

```

代码 1: 子块乘加 (节选)

对每个 k , 先完成当前轮所需的 $A_{i,k}$ 、 $B_{k,j}$ 阻塞收发, 再调用 `gemm_accum`; 一轮结束后再进入下一 k , 无计算与通信重叠。非根进程在 `gather_C_blocks` 中向 `rank 0` `MPI_Send` 本地 $C_{i,j}$, `rank 0` 依次 `MPI_Recv` 并写入全局 C , 作为计时段的收尾。

(二) 计时与五轮基准实验

计时段起点为全体进程在 `scatter` 前经 `MPI_Barrier` 同步后、以 `rank 0` 的 `MPI_Wtime` 记下的全局原点; 终点为本地 $C_{i,j}$ 在 `gather_C_blocks` 中汇聚完成之时。根进程在计时外完成 A, B 初始化。脚本 `bench_matmul_mpi.sh` 每轮依次调用 7 个进程数配置; 五轮数据写入同一 CSV, 列 `rep` 区分轮次, 事后对同一 (N, p) 的 5 个 `time_sec` 取平均作为 T_p 。

(三) 时间线分析实验

脚本 `bench_matmul_timeline.sh` 按进程数逐个测试, 亦可用 `--p` 只跑单个配置。每次运行开启 `MATMUL_EVENTS=1`, 不修改 `matmul_bench.csv`; 事件按 `rank` 写入 `results/events/`, 由 `plot_timeline.py` 绘制 Gantt 图, 其中 `--batch` 可一

次生成 `plots/timeline_p<p>.png`。对 p 较大时脚本限制图高并按 **rank** 稀疏标注纵轴，避免图幅过高。事件仅覆盖显式记录的 `pack/send/recv/compute/alloc`；各 **rank** 上事件时长之和与墙钟接近，未出现大块“未记录时间”，说明时间线中的空白主要表示该时段无上述操作，而非遗漏了隐藏的 **MPI** 等待。

（四）程序编译与运行

使用 `mpicc` 编译；`make` 等价于执行 `mpicc -O3 -std=c11 -o matmul_mpi matmul_mpi.c`。基准示例：`./bench_matmul_mpi.sh 1 --fresh`。时间线示例：`./bench_matmul_timeline.sh --p 64 --fresh`。单次手工运行：`mpirun --use-hwthr -np 64 ./matmul_mpi 8000`。可执行文件与脚本位于 `h3/`，结果位于 `h3/results/` 与 `h3/plots/`。

四、实验结果与分析

（一）五轮基准实验结果

表 5 给出 $N = 8000$ 下五轮 `bench_matmul_mpi.sh` 的 `time_sec` 算术平均及由此得到的 S_p 、 E_p ，其中 $T_1 = 4216.55\text{s}$ 。五轮之间同一 p 的波动较小； $p = 1$ 的标准差略大，其余配置多在 2s 以内，说明批量脚本下结果可重复。

表 5 **MPI** 矩阵乘法性能结果 ($N = 8000$ ，五轮平均，单位：秒)

进程数 p	平均耗时 T_p	加速比 S_p	并行效率 E_p
64	67.48	62.49	0.976
32	87.19	48.36	1.511
16	170.76	24.69	1.543
8	468.01	9.01	1.126
4	933.33	4.52	1.129
2	5717.96	0.74	0.369
1	4216.55	1.00	1.000

图 4、图 5 由 `plot_matmul_bench.py` 根据五轮 `results/matmul_bench.csv` 绘制；横轴均为 **MPI** 进程数，取以 2 为底的对数刻度。

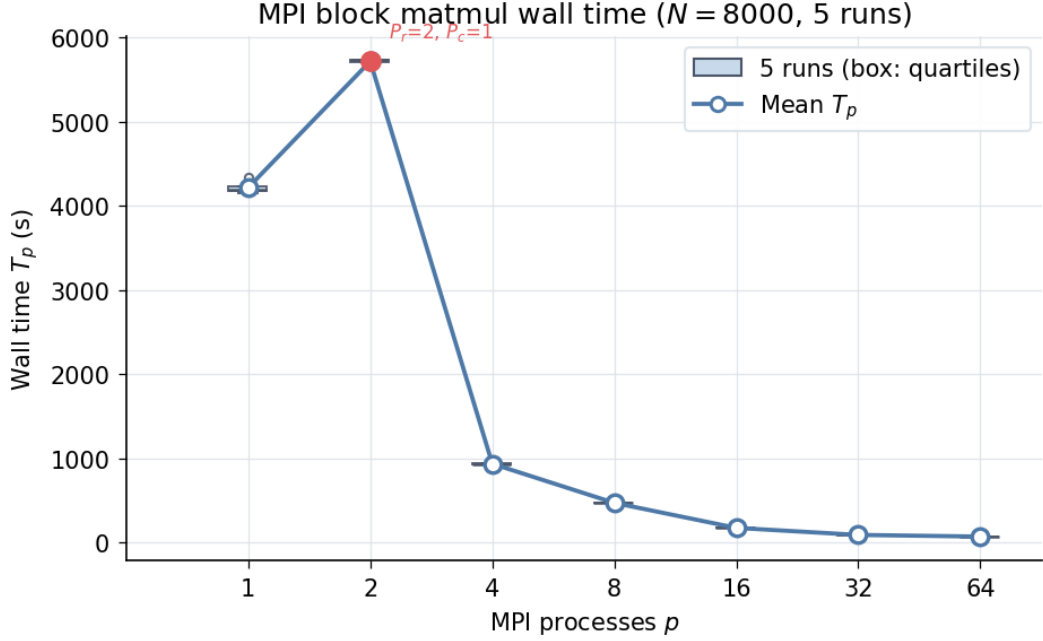


图 4 不同进程数下的运行时间 T_p ($N = 8000$, 箱线为五轮分布, 折线为均值)

由图 4 可见, $p \geq 4$ 时 T_p 随进程数增加而下降, 箱线较窄表明五轮重复稳定; $p = 2$ 处箱线位置突增, 与 $p = 1$ 接近, 直观呈现退化网格效应。

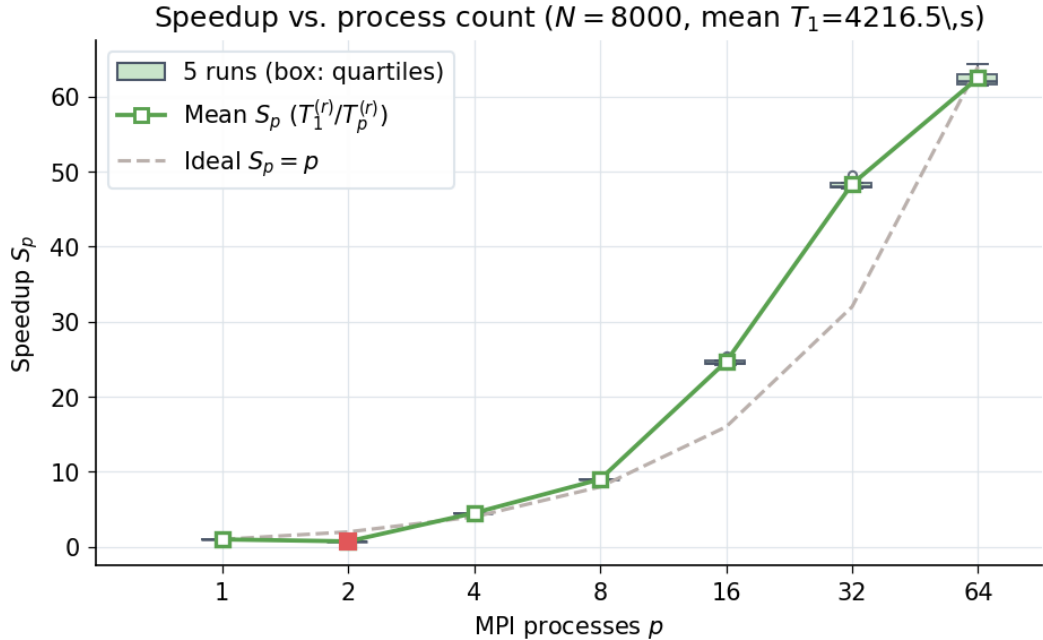


图 5 加速比随进程数的变化 (箱线为每轮 $T_1^{(r)}/T_p^{(r)}$, 折线为平均 S_p , $T_1 = 4216.55$ s)

图 5 中, 从 $p = 64$ 降至 $p = 4$ 时 S_p 由约 62.5 降至约 4.5, 整体呈可扩展趋势; $p = 64$ 时 $E_p \approx 0.98$, 并行资源利用较充分。 $p = 16$ 、 $p = 32$ 处平均加速比高于理想直线, 与前述 T_1 基线含义有关。讨论强扩展时宜以 $p \geq 4$ 为主; $p = 2$ 与 $p = 1$ 需结合进程网格形

状单独解释，下节结合时间线进一步说明。

$p = 32$ 、 $p = 16$ 时 $E_p > 1$ ，即实测 S_p 高于理想直线 $S_p = p$ ，并不必然表示串行实现存在错误。基准中的 T_1 来自 `mpirun -np 1` 下单进程完成整块 8000×8000 乘加，工作集约 **1.5 GiB**，cache 局部性很差； $p = 16$ 时子块约 2000×2000 ，每进程算量约为整体的 $1/16$ ，但单核算子访存更友好，再叠加多进程并行，便可能出现相对 T_1 的超线性加速。这反映的是划分带来的局部性变化与 T_1 作为基线的含义，而非理想模型 $S_p = p$ 在每个进程数下都应严格成立。

$p = 2$ 时 $T_p \approx 5718\text{s}$ ， $S_2 < 1$ ，在图 4、图 5 中明显偏离 $p \geq 4$ 的趋势，其成因见下节时间线分析。

（二）时间线实验与计时段行为

时间线实验与五轮基准分离：`bench_matmul_timeline.sh` 对每个 p 运行一次并开启 `MATMUL_EVENTS=1`，由 `plot_timeline.py --batch` 生成图 6–12。与表 5 对照，`matmul_bench_timeline.csv` 中墙钟与基准相差在 1% 以内，例如 $p = 64$ 分别为 67.59s 与 67.48s ，说明事件记录对 T_p 干扰很小。读图时需注意：色块仅对应 `pack/send/recv/compute/alloc`；`recv` 在阻塞语义下包含等待；各 `rank` 之间除计时段起点外无屏障，故空白可能表示尚未进入下一段记录操作，也可能表示已完成本地乘加后等待其它 `rank`，或处于未插桩时段。下文按基线、退化网格、近方阵扩展与高进程数四条线索展开。

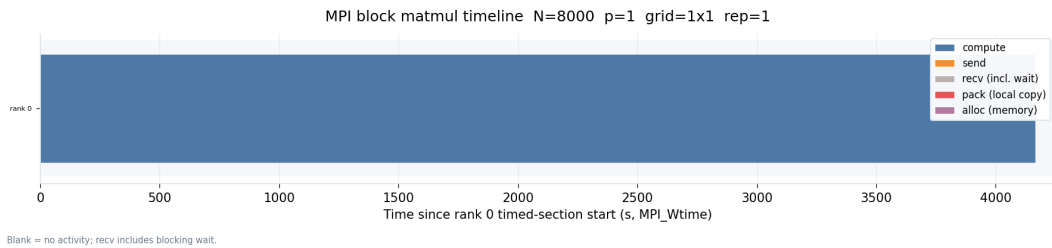


图 6 $p = 1$ (1×1 网格)：计时段即整表乘加

图 6 为单进程基线：仅有 `rank 0` 一行，计时段 4166s 中 `k_loop` 的 `compute` 约 4165s ，`scatter/gather` 各不足 1s ，且无 `send/recv`。这与 T_1 的含义一致——计时几乎等于整表朴素乘加；后文出现的 `send/recv` 条带均可与此对照。

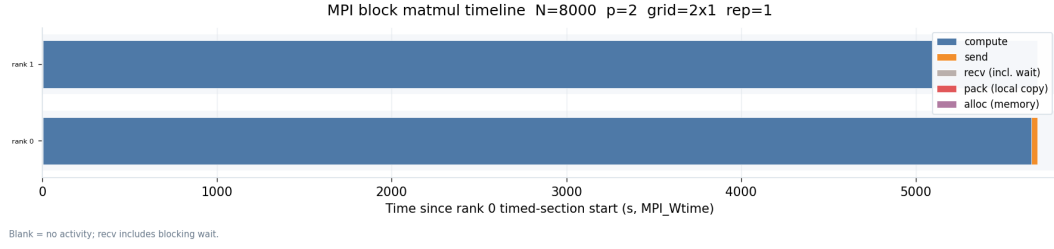


图 7 $p = 2$ (2×1 网格): 计算主导, 但单核算子效率低

图 7 对应退化 2×1 网格: 两 rank 的 k_loop 计算分别约 5693s 与 5658s, MPI 收发合计仅数秒; rank 1 在 gather 阶段的 send 约 35s 位于末尾, 但仍远小于计算时间。事件结束时刻与墙钟 5696s 相差不足 1s, 说明性能变差并非漏记通信, 而是每 rank 本地 $gemm_accum$ 吞吐远低于 $p = 1$: 算量约为整表一半, 耗时却增加约 37%, 折合约 45 MFLOP/s, 而 $p = 1$ 约为 123 MFLOP/s。狭长子块与 $P_c = 1$ 使按 k 的乘加退化为单轮, 两进程又同机争抢带宽, 故 $p = 2$ 不宜与 $p \geq 4$ 一并讨论强扩展。

当 $P_r \times P_c$ 接近方阵时, 时间线形态与表 5 中 $p = 4 \sim 16$ 的可扩展段一致; 以下依次给出 $p = 4, 8, 16$ 的时间线。

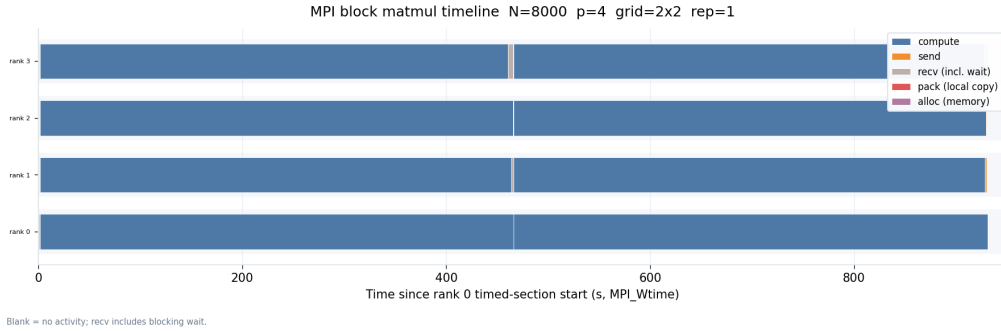


图 8 $p = 4$ (2×2 网格): 四 rank 计算时长接近, rank 0 承担分发与汇聚

图 8 中墙钟约 931s, 四 rank 乘加时间集中在 920 ~ 929s, 负载不均小于 1%。rank 0 在两端完成子块分发与汇聚, gather 阶段 $recv$ 略长于 $scatter$; $P_c = 2$ 使每个 rank 上出现两段 compute 带, 对应两轮 k 乘加。

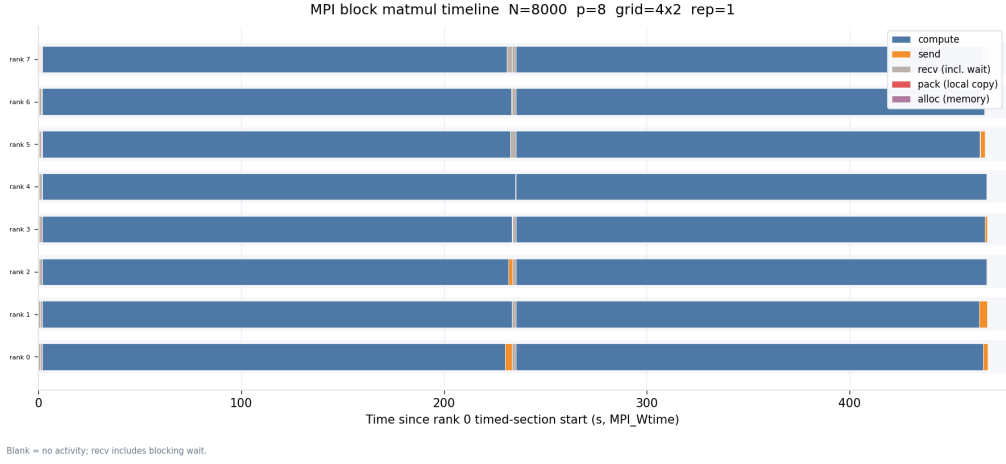


图 9 $p = 8$ (4×2 网格): $P_c = 2$, 每个 rank 可见两轮 k 乘加

图 9 中墙钟约 468s, 八 rank 的 k_loop 计算约 458 ~ 465s. 4×2 网格下通信条带仍呈零星分布, 未见某 rank 长期停留在 `recv`, 说明阻塞等待尚未主导墙钟。

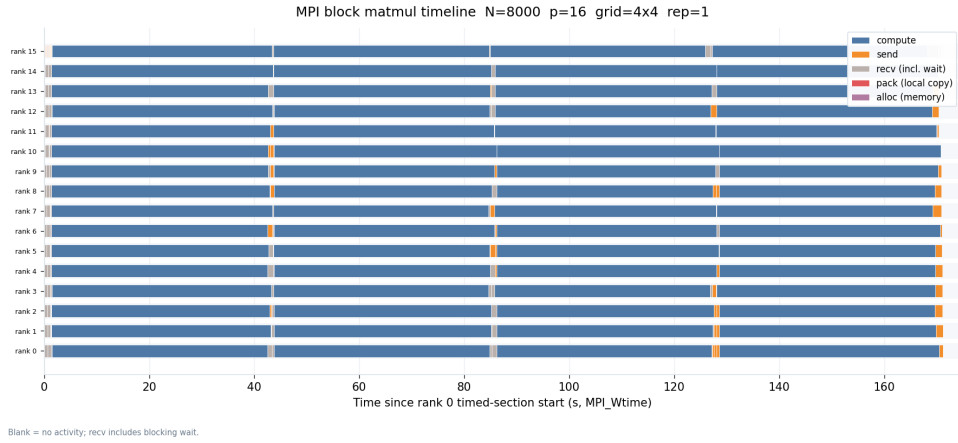


图 10 $p = 16$ (4×4 网格): 负载均衡, 计算条带进一步缩短

图 10 中墙钟约 171s, $P_c = 4$ 使每个 rank 的计算条带切为四段。通信碎片占比仍远小于 `compute`, 与 $E_{16} > 1$ 相呼应: 相对 T_1 而言, 子块变小改善访存, 并行乘加成为主要收益来源。

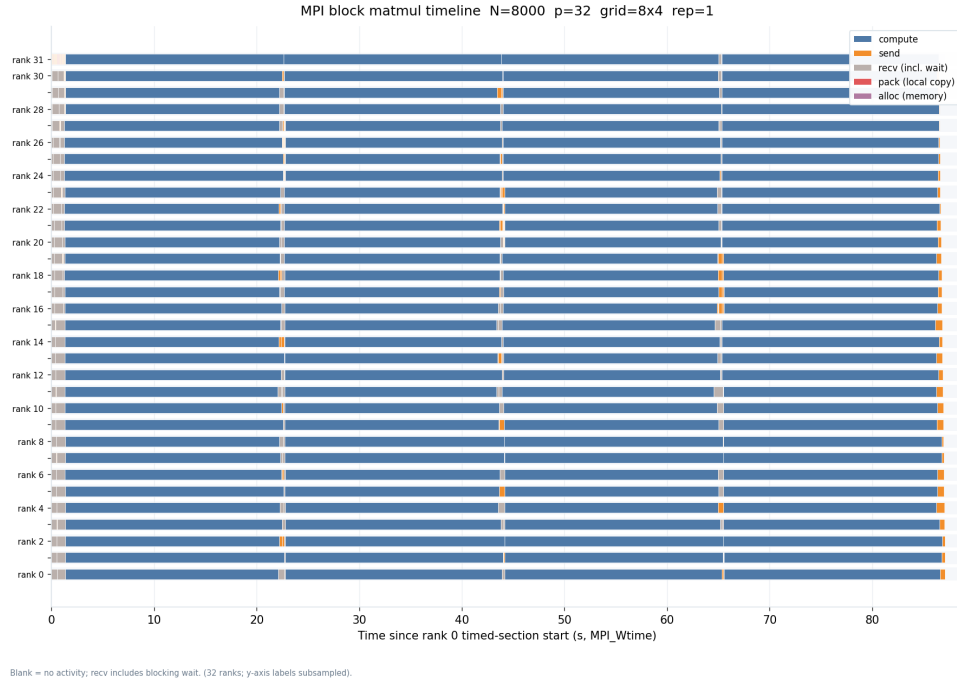


图 11 $p = 32$ (8×4 网格): 仍较均衡, 墙钟接近最慢 rank 的计算时间

图 11 中各 rank 的 k_loop 计算约 $82.7 \sim 85.1s$, 墙钟 $87s$ 与最慢计算接近; rank 0 的 scatter/gather 合计约 $1.8s$, 未成为瓶颈, 16 条 rank 轨迹几乎平行, 说明 8×4 划分下仍处良好强扩展区间。

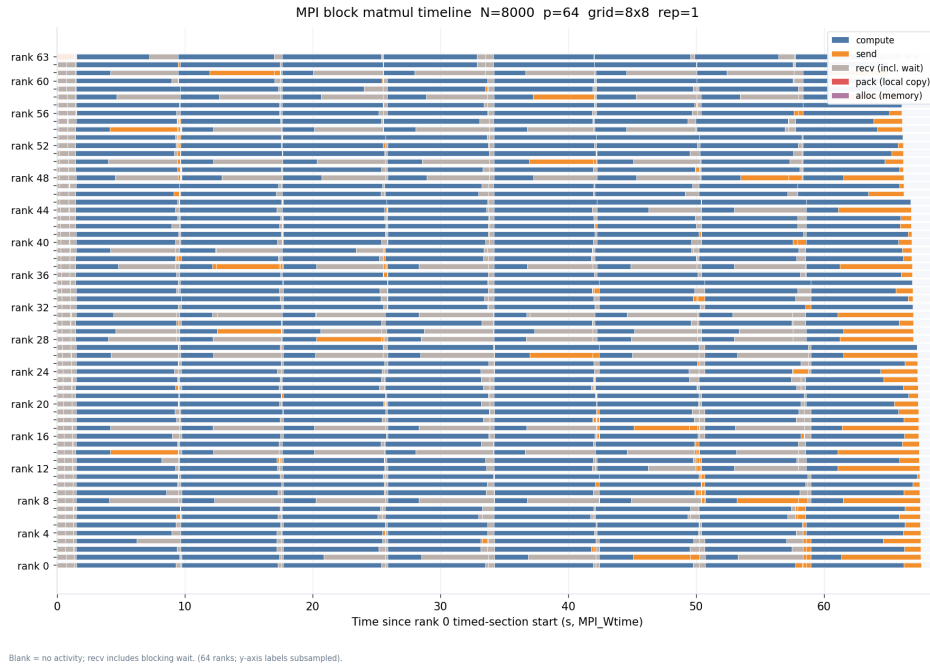


图 12 $p = 64$ (8×8 网格): rank 0 通信事件密集, 工作 rank 计算与等待差异拉大

图 12 呈现不同图景: rank 0 事件条数远多于其它 rank, 对应向全体进程分发子块并依次接收 $C_{i,j}$; 其 k_loop 计算约 $56s$, scatter 与 gather 合计约 $5.4s$, 计时段在 rank

0 结束，墙钟 67.6s。工作 rank 乘加时长分散在约 20 ~ 65s 之间，部分 rank 的 `recv` 累计可达数十秒；快 rank 在乘加结束后至 `gather` 前还可能存在未插桩空档。相对 $p = 32$ 仍有约 1.3 倍加速，但串行汇聚与进程不同步值得在更大规模下优化。

综上，七幅时间线从“纯计算”到“计算 + 周期性 k 轮通信”再到“高 p 下 rank 0 枢纽化”，与表 5 及图 4 的趋势相互印证： $p = 2$ 的异常源于划分与单核算子效率而非隐藏通信； $p = 4 \sim 32$ 以均衡的本地乘加为主； $p = 64$ 则在保持较短墙钟的同时暴露出汇聚与负载不均。本程序全程使用阻塞式 `MPI_Send/MPI_Recv`，计时段仅在起点使用 `MPI_Barrier`，未在阶段间强制同步，故时间线适合观察各 rank 真实先后关系，而非理想同步并行模型。